🖎 WPF Project (DataExplorerIg)

Purpose

This WPF project demonstrates a **data exploration dashboard** built with **Infragistics WPF data controls and templates**. It simulates real-world enterprise use cases such as shipment tracking, compliance dashboards, and hierarchical data navigation.

Key Code Features

- **XamDataGrid / DataPresenter**:
  - Hierarchical layouts with multiple bands.
  - Custom cell templates for editors (text, combo, numeric).
  - Grouping, sorting, and filtering enabled.

- **XamDataTree / Hierarchical Menu**
  - For tree-menu features for data selection.
  - Drag drop capability to add selected records to data view for further processing.

- **ListView Template**
  - For **simpler tabular data presentation**, used alongside Infragistics grids to extend demonstration of automation across both native WPF and infragistics controls.

- **XamTabControl & DatatPresenter**
  - For **multi-view navigation**, allowing users to switch between grid, chart, and editor panels seamlessly.
  - For structured data visualization.

- **Editors & Templates**:
  - TextBox, ComboBox, and DatePicker embedded inside grid cells.
  - Custom styles applied for validation scenarios.

- **Data Binding**:
  - JSON-based data source (via Newtonsoft.Json).
  - Simulates import/export records with nested structures.

- **UI Composition**:
  - Multiple views (grid, chart, editor panels).
  - Trial Infragistics assemblies integrated (v23.2.290).

Test Project (DataExplorerIg.Tests)

Purpose

The test project validates the WPF app using **FlaUI automation framework**. Its coverage includes many relevant real time scenarios to assert that even **complex Infragistics controls and templates** can be automated reliably using the given automation framework and design.

Actual Test Scenarios in Code

1. Application Launch & Window Validation

- **Scenario:** The test suite launches the DataExplorer WPF application.
- **Validation:** Confirms that the **main window** is created and accessible.
- **Capability Demonstrated:** Reliable app startup automation and window discovery using FlaUI.

2. XamDataGrid Row Selection

- **Scenario:** The test locates the **Infragistics** XamDataGrid control.
- **Action:** Selects the **first row** in the grid.
- **Validation:** Ensures the row is successfully highlighted/selected.
- **Capability Demonstrated:** Automation of complex Infragistics grid interactions.

3. TextBox Editor Interaction

- **Scenario:** After selecting a grid row, the test finds an **embedded TextBox editor**.
- **Action:** Inputs text into the editor (e.g., simulating user data entry).
- **Validation:** Confirms the editor accepts input and updates the UI state.
- **Capability Demonstrated:** End-to-end automation of **inline editors inside grids**.

4. XamTabControl Navigation

- **Scenario:** The test locates the **Infragistics** XamTabControl.
- **Action:** Switches between tabs (e.g., Grid → Chart → Editor).
- **Validation:** Confirms that controls inside each tab are accessible and functional.
- **Capability Demonstrated:** Automated navigation across **multi-view dashboards**.

5. ListView Item Selection

- **Scenario:** The test identifies a standard WPF **ListView** control.
- **Action:** Selects items from the list.
- **Validation:** Confirms the selected item is highlighted and accessible.
- **Capability Demonstrated:** Shows automation across both **native WPF controls** and **third-party Infragistics controls**.

6. Drag & Drop from Tree to Grid

- **Scenario:** Automates dragging an item from a **Tree control** and dropping it into the XamDataGrid.
- **Validation:** Confirms the item is successfully added to the grid and displayed in the correct row.
- **Capability:** Demonstrates handling of **complex user gestures** (drag & drop) in automation.

7. Group-By Feature in Grid

- **Scenario:** Automates grouping rows in the XamDataGrid by a specific column (e.g., *Status* or *Category*).
- **Validation:** Confirms grouped rows are displayed correctly under expandable headers.
- **Capability:** Validates advanced grid features like **grouping, hierarchical display, and expand/collapse automation**.

🔑 Capability Showcase

Capability	Demonstration
Complex Control Automation	FlaUI selectors handle Infragistics controls, editors and templates
Hierarchical Data Testing	Automated expansion and validation of nested bands
End-to-End Workflow	Grid edits → validation → data updates
Robust Error Handling	Null selectors, multiple instances, screenshot logging, invocation through parameterized scripts
Reusable Framework	NUnit-based test suite, extendable for enterprise dashboards

🚀 Client-Facing Value Addition

1. Automation of Complex Infragistics Controls

- **Value:** Many enterprise dashboards rely on Infragistics controls (XamDataGrid, XamTabControl, DataPresenter) because of their rich functionality.
- **Impact:** By proving automation feasibility with FlaUI, this POC shows clients that even **the most complex, third-party UI components** can be validated automatically.
- **Benefit:** Reduces manual QA cycles, accelerates release timelines, and ensures compliance-ready dashboards.

2. End-to-End Workflow Validation

- **Value:** Tests simulate **real user journeys**—from selecting rows in a grid, editing inline text, navigating tabs, dragging items from a tree into a grid, and grouping data by columns.
- **Impact:** This demonstrates automation beyond simple clicks, covering **multi-step workflows** that mirror actual business processes.
- **Benefit:** Clients gain confidence that automation can validate **business-critical scenarios** like shipment tracking, compliance grouping, and dashboard navigation.

3. Advanced User Interactions (Drag & Drop, Grouping)

- **Value:** Automating drag & drop from a tree into a grid and validating group-by features shows mastery of **complex gestures and advanced grid functionality**.
- **Impact:** These are high-value scenarios in enterprise apps (e.g., dragging shipment items into containers, grouping by compliance status).
- **Benefit:** Clients see that automation can handle **real-world, high-complexity interactions**, not just basic UI clicks.

4. Cross-Control Coverage

- **Value:** The test suite covers both **Infragistics controls** and **native WPF controls** (ListView, TextBox).
- **Impact:** This breadth ensures automation is not limited to one vendor's toolkit.
- **Benefit:** Clients can extend the same framework across **all parts of their application**, reducing tool fragmentation.

5. Robustness & Reliability

- **Value:** Error handling (null selectors, multiple app instances, screenshot logging) ensures tests are **resilient in production pipelines**.
- **Impact:** Reduces false positives and makes automation trustworthy.
- **Benefit:** Clients can integrate these tests into CI/CD pipelines with confidence, improving release quality.

6. Strategic Business Advantage

- **Value:** By automating complex dashboards, clients can **shorten QA cycles, reduce costs, and improve compliance accuracy**.
- **Impact:** Faster delivery of validated dashboards means quicker insights for decision-making.
- **Benefit:** Positions the client as a **technology-forward organization** capable of delivering reliable, compliance-ready solutions at scale.

✅ Summary

This POC is a **capability showcase** to affirm that the consolidated features and framework can be scaled to deliver **enterprise-grade automation** for complex WPF applications, covering advanced Infragistics controls, native WPF elements, and real-world workflows. This can translate into **reduced QA effort, faster releases, and higher confidence in compliance-critical deliveries.**